

Due by end of day (11:59pm) Thursday April 2

Project #2: The Tipping Point

§ Introduction

For this project, you will explore, albeit briefly, PostgreSQL's query optimizer (v17.6) and how it optimizes SQL queries. (Let us call “PostgreSQL” as “Postgres”, for short.) The Postgres server you will use is the default one set up on the “PRISM” eecs.yorku.ca domain.

This hosts the database stl. In particular, your task is to explore when the optimizer switches from one plan to another for the same templated query, as reduction factors are changed. These are the “tipping points” in the plan space.

§ The StL Data Warehouse

The StL data warehouse was for St Lawrence Booksellers, Inc. It recorded sales by customer and book from the years 2012 to 2017. StL was a small, boutique web-based bookseller.

¶ The StL Data Warehouse Schema

The schema of StL consists of three tables.

Customer		Book		Purchase	
<u>custid</u>	INTEGER	<u>bookid</u>	INTEGER	<u>custid</u>	INTEGER
country	VARCHAR(20)	language	VARCHAR(20)	<u>bookid</u>	INTEGER
province	VARCHAR(20)	genre	VARCHAR(20)	<u>whenp</u>	DATE
city	VARCHAR(20)	publisher	VARCHAR(40)	qnty	INTEGER
		price	DECIMAL(5, 2)	sale	DECIMAL(6, 2)

The underlined attributes show the primary keys. In the language of data-warehousing, Customer and Book are dimension tables, and Purchase is the fact table. Customer has 100,000 rows, Book has 50,000 rows, and Purchase has 9,997,204 rows. There is a foreign key from Purchase to Customer and from Purchase to Book.

In Purchase, custid, bookid, and whenp are the dimension columns. The measure column is sale. It reports how much the sale was for. Each sale is equal to the book's price × qnty. The column qnty indicates how many copies of this book was bought in that given purchase. In most cases, qnty = 1. (Note that sale has already accommodated qnty! In your queries, you do not have to multiply by qnty.)

Since StL is a web-based book-seller, we mail the books to the customers' addresses. So one can think of “customer” as a location. Call that dimension location then. That dimension offers a natural roll-up: country > province > city > custid.

The next “dimension” is type, and is a characteristic of table Book. Books have three attributes of interest:

- *language* (what language it is written in);
- *genre* (what category it is in, e.g., humor, politics, etc.); and
- *publisher* (what company publishes it).

These form three dimensions on Book, and offer another natural cube, treating Book itself as a small fact table.

The last dimension is time, which is represented by the attribute *whenp* in Purchase. It is of type DATE. So it does not show the time of a purchase, but reports the day of purchase. Notice there is no table Time, but we could pretend there is a virtual table for Time. E.g.,

```
Time
year  YEAR
month MONTH
day   DAY
```

Thus, the time dimension provides for a natural roll-up. Standard SQL provides many functions for dealing with DATE and TIMESTAMP values. In fact, we can roll up whenp in many ways.

There are no NULL values in the data warehouse.

¶ Indexes

The clustered indexes are on the primary keys as follows.

- Customer:
 - custid
- Book:
 - bookid
- Purchase:
 - custid, bookid, whenp

Table Purchase has two additional unclustered indexes:

- bookid
- whenp

§ Accessing PostgreSQL on PRISM

We are running a PostgreSQL server on PRISM, the EECS network. It is managing the StL-DW database, as described above.

The database is accessible via the PostgreSQL client shell, psql on any of the PRISM / EECS machines (departmentally managed linux machines in the eecs.yorku.ca) domain; e.g., red.eecs.yorku.ca.

¶ General Notes

- We will most often abbreviate “PostgreSQL” as “Postgres,” simply because it is easier to say and read.
- ([Postgres vs PostgreSQL — What is the difference? | TablePlus](#))
- “PRISM” is the nickname of EECS's network and collection of computers.
- The Postgres(ql) server for EECS classes
 - runs on host machine db.eecs.yorku.ca and
 - the present version is 17.6.
- All machines on the PRISM LAN (local area network) are hidden behind a firewall. You must go through one of the gateway machines — e.g., red.eecs.yorku.ca or ruby.eecs.yorku.ca — to access any of them from outside of the EECS LAN.
- *nix is the general term for any operating system based in principle on unix. This covers, of course, any of the variants of unix, linux, and macOS (for which the core is a unix kernel called darwin).

¶ How to connect to a database

You need a program to set up a connection to the database server — and specify the database that the server manages to which you want to talk — to be able to “interact” with the data (in that database that that database server is managing). The only way to interact with a relational database is via the query language, as we know. For relational databases, the (nearly universal) standard query language is SQL.

It is useful to be able to connect directly to a database as a user to issue (SQL) commands and queries directly. In operating systems, a general command-line program where the user can issue commands directly is called a shell. (The term shell is used more generally for any interactive, command-line program interface to a system.) The psql shell is the main tool for interacting with a database managed by a PostgreSQL server.

Most database systems are a client-server configuration: the DBMS (data-base management system) runs as a server; and applications that use and manipulate the data managed by the DBMS are clients. Most often, a database client does not run on the same machine as the server. (A DBMS server may have many, many active clients at any given time.) This will commonly be the case for psql clients.

Are there non-shell, general-purpose clients for Postgres, like pretty GUI (graphical user interface) ones? Yes, of course. But we do not really need them here.

¶ The PSQL shell client

All the machines in PRISM have the psql client installed. To connect to your (or some) database hosted by DB, open a shell on your favourite PRISM machine and type, say,

```
% psql
```

and you will be in the client shell! How do you navigate in the psql shell? Well, you would need to learn that.

- [PostgreSQL: Documentation: 17: psql](#)
- [psql command line tutorial and cheat sheet | postgres | tomcam.github.io](#)
A nice, concise tutorial on using psql, courtesy of *Tom Campbell*.

Note that there are two ways of using psql:

- Starting the psql shell and issuing psql commands in the shell session.
- Issuing psql commands from your *nix shell.

You will find examples of these below.

¶ Postgres Accounts in EECS

By EECS's set up, your Postgres user account is set up to be the same name as your EECS account name. And things are set up so when you “log in” to “Postgress” (that is, authenticate with the client), you are automatically authenticated. (This is done via Kerberos. See below.)

By default, Postgres creates a database with the same name as your user name. This is the database to which you connect by default. Feel free to play around!

¶ Accessing the StL-DW database

For this project, you want to connect to the database stl, however. The StL-DW tables are housed there under a schema named stl. E.g.,

```
% psql -d stl
```

The three tables are in a schema named stl. So to access them, one needs to prepend “stl.” in front of the table name. E.g.,

```
stl => select count(*) from stl.purchase;
```

The StL-DW is reasonably large. Queries can take a few seconds to a minute or so to execute. Be patient. For real-world data warehouses, query execution times can be hours.

¶ Kerberos Authentication at EECS

You need an active Kerberos ticket to be able to authenticate with psql. (Otherwise, you get receive an authentication error when you try to connect.) A ticket ought to be created automatically whenever you log into an EECS machine, so this generally will be seamless.

But if you do get an authentication error, issue a new Kerberos ticket:

```
% kinit
```

And then connecting should work for the rest of that “session”.

See [Kerberos for EECS Linux Systems \[Technical Database\]](#) for more.

§ Profiling Query Plans

We shall use the explain command to explore query plans that the Postgres optimizer creates.

To prepare and explain a query from a linux shell on a PRISM machine, issue the command

```
% psql -d stl -c "explain 'query text here;'" -o 'output_file'
```

with 'query text here' (so without the single quotes) replaced by the SQL of your query and 'output_file' replaced by the name of the file into which you want the output dumped.

One can have psql read a file to execute it:

```
% psql -d stl -f 'input_file.sql'
```

with 'input_file.sql' replaced by the name of your file. This is a standard way to execute a query. Note that if your SQL query in the file had the keyword explain before it, then psql would then explain the query instead.

If you have an SQL in a file already that you want to explain, say, my.sql, one work-around under the csh or tsch shell would be

```
% psql -d stl -c "explain `cat my.sql` "
```

And under the bash or zsh shell,

```
% psql -d stl -c "explain $(cat my.sql)"
```

One can issue the explain command from within a psql session — and so with the database connection already at hand — as well:

```
% stl => explain 'query text here';
```

with 'query text here' replaced by the SQL of your query.

To output the query plan into a file in this case, say:

```
% stl=> \o output_file
```

```
% stl=> explain 'query text here';
```

It is possible to check the accuracy of the planner's estimates by using explain's analyze option. With this option, explain actually executes the query, and then displays the true row counts and true runtime accumulated within each plan node, along with the same estimates that the plain explain shows.

To be able to understand the query plan generated by PostgreSQL's query planner, please refer to:

- [“PostgreSQL \[17\]: Documentation: EXPLAIN”](#) and
- [“PostgreSQL \[17\]: Documentation: §14.1. Using EXPLAIN.”](#)

§ The Tasks

¶ A. Explaining the Plan [5pts]

Write an interesting SQL query for the StL-DW database. Your query

should involve all three tables in the schema, and

include some non-join predicates.

For your query, provide the following.

1. Your SQL query.
 - a. A plaintext file named A-query.sql with your query.
 - b. In a plaintext file named A-report.txt, explain in clear, plain English what your query asks.
2. The query plan that Postgres generates for your query. Generate the plan annotated with runtime statistics (i.e., explain analyse.)
 - a. A plaintext file named A-plan.txt with the query plan as dumped from Postgres's explain analyses
 - b. Explain in clear, plain English what the query plan does.

- i. State what the access paths are and the algorithms chosen (e.g., hash join) for the different operators in terms of what we discussed in class.
 - ii. If the query plan is using any techniques / algorithms not introduced in class, point those out, and give a reasoned speculation what they do.
3. Performance analysis.
 - a. In A-report.txt, interpret the runtime performance versus what the optimizer estimated in the plan.
 - b. Point out any significant deviations between the estimated and runtime costs.

¶ B. Variants [5pts]

The two queries below are variants of each other; that is, they are the same except for a small difference in the where clause. What is interesting about them is that the optimizer produces different query plans for each of them.

```
select sum(sale) as total
from stl.purchase
where whenp between '01/01/2012' and '01/31/2012';

and

select sum(sale) as total
from stl.purchase
where whenp between '01/01/2012' and '10/31/2015';
```

Your job is to find a pair of query variants as follows:

- The two queries should be variants of each other.
- Each query should involve all three tables in StL-DW (so, involve two joins).
- The two queries should involve the same predicates, but with different values (as in the example above).
- The query plans generated for each should differ significantly.

Refer to “[PostgreSQL execution plan operations](#)” of “Use the Index, Luke! A Guide to Database Performance for Developers.”

Of course, there are many ways you can design your query variants. The challenge is to have different plans result for the two variants, with a minimal difference between the variant queries. If you fail to generate different plans for your queries, you still must provide an explanation on what you tried, and why you think it failed to result in differing query plans.

For your query variants, provide the following.

1. The source SQL of your two variant queries
 - a. in plaintext files B-one.sql and B-two.sql, respectively.
2. The query plans that Postgres generates for your two queries
 - a. in plaintext files B-plan1.txt and B-plan2.txt, respectively.
3. Explain in clear, plain English in a plaintext file named B-report.txt
 - a. what each query plan does (as in part A.),
 - b. explain how the two plans differ, and

- c. surmise why the plans differ based on how the two query variants differ.

¶ C. Helping the Optimizer [5pts]

SQL is a declarative language. But it does not mean that we never need to “help” the optimizer by writing a more “efficient” query. This is the challenge here.

Rewrite the following query so the rewritten query has the same meaning, but which leads to having better query efficiency (a better query plan), and explain why this happens. The rewritten must be correct; that is, the original query and your rewritten version must return exactly the same result set.

```
select custid, language, genre, publisher
```

```
from stl.Customer C,
```

```
  ( select language, genre, publisher
```

```
    from stl.Book B
```

```
    group by language, genre, publisher
```

```
    having count(*) > 10
```

```
  ) as B
```

```
where custid < 500
```

```
except
```

```
select custid, language, genre, publisher
```

```
from stl.Book B,
```

```
  ( select custid, bookid
```

```
    from stl.Customer, stl.Book
```

```
    where custid < 500
```

```
    except
```

```
    select distinct custid, bookid
```

```
    from stl.Purchase
```

```
    where custid < 500
```

```
  ) as N
```

```
where B.bookid = N.bookid;
```

The query is asking to find customers whose ID is less than 500 who have bought all the books that StL has available within a language-genre-publisher category that contains more than ten books. The query as written is very time consuming. Writing an efficient query is the challenge here. Consider using aggregation to count the number of books there are in each language-genre-publisher category, and likewise for the number of books each customer bought in each language-genre-publisher category. (Note that using the with clause could be quite helpful.)

For your rewritten query, provide the following.

1. Your rewritten SQL query.
 - a. in a plaintext file named C-rewrite.sql.
2. The query plan that Postgres generates for your rewritten query
 - a. in a plaintext file named C-plan.txt.

For full credit, your query must be correct. And your query must execute efficiently, in less than a couple of minutes.

§ Deliverables

Make a directory named tipping for your project. Within the directory, have the following files as described above:

- A-query.sql,
- A-plan.txt,
- A-report.txt,
- B-one.sql,
- B-two.sql,
- B-plan1.txt,
- B-plan2.txt,
- B-report.txt,
- C-rewrite.sql, and
- C-plan.txt.

Your SQL must be well formatted, readable, and understandable. (E.g., no line breaks, no indentation is not readable.)

Keep your report simple; e.g., each report under 1,000 words. Each file must be a plaintext file (ASCII or utf-8). So no .pdf, .doc, .docx, or other file formats are permitted. (Note that any non-plaintext files will simply be ignored.)

Submit the directory as follows.

% submit 4411 tipping <file> to submit from the command line

Or you can use <https://webapp.eecs.yorku.ca/submit>

§ Frequently Asked Questions (FAQ)

Hints, clarifications, and answers to frequently asked questions regarding the project will accumulated here.

Q. For doing the project, could we install a Postgres client (e.g., psql) on our own computer to connect to the Postgres server at db.eecs.yorku.ca.

A. In principle, this is possible. However, most organizations limit that database clients connecting to their database servers be within their firewall. That is the case for EECS too. So, no.

This means you will need to use a PRISM computer (e.g., ruby.eecs.yorku.ca) to do the project. You can always use EECS's remote lab to connect to a PRISM computer from outside to do work.

Acknowledgements :

This project description mainly comes from Professor Parke Godfrey.